
UNIT 4 SOLVING RECURRENCES

Structure

- 4.0 Introduction
- 4.1 Objectives
- 4.2 Recurrence Relation
- 4.3 Methods for Solving Recurrence Relation
 - 4.3.1 Substitution method
 - 4.3.2 Recursion Tree Method
 - 4.3.3 Master Method
- 4.4 Summary
- 4.5 Solution to check your progress
- 4.6 Further Reading

4.0 INTRODUCTION

Complexity analysis of iteration algorithms is much easier as compared to recursive algorithms. But, once the recurrence relation/equation is defined for a recursive algorithm, which is not difficult task, then it becomes easier task to obtain the asymptotic bounds (θ , O) for the recursive solution. In this unit we focus on recursive algorithms exclusively. Three techniques for solving recurrence equation are discussed: (i) Substitution method (ii) Recursion Tree Method and Master Method. In the substitution method, we first guess an asymptotic bound and then we prove whether our guess is correct or not. In the recursion tree method a recurrence equation is converted into a recursion tree comprising several levels. Calculating time complexity requires taking a total sum of the cost of all the levels. The master method requires memorization of three different types of **cases** which help to obtain asymptotic bounds of many simple recurrence relations.

4.1 OBJECTIVES

After going through this unit you will be able to

- Define recurrence relation
- Construct recurrence relation of simple recursive algorithms
- List the techniques used to solve recurrence relation
- Solve the recurrence relation through , Substitution, Recurrence tree & Master methods.

4.2 RECURRENCE RELATION

We often use a *recurrence relation* to describe the running time of a recursive algorithm. A *recursive algorithm* can be defined as an algorithm which makes a recursive call to itself with smaller data size. Many problems are solved

Solving Recurrence

recursively, especially those problems which are solved through Divide and Conquer technique. In this case, the main problem is divided into smaller sub-problems which are solved recursively. Quick Sort, Merge Sort, Binary search, Strassen's multiplication algorithm are formulated as recursive algorithms. These problems will be taken up separately in the next block.

Like all recursive functions, a recurrence relation also consists of two steps: (i) one or more initial conditions and (ii) recursive definition of a problem

Example 1: A Fibonacci sequence $f_0, f_1, f_2, \dots$ can be defined by the recurrence relation as:

$$f_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ f_{n-1} + f_{n-2} & \text{if } n \geq 2 \end{cases}$$

1. **(Basic Step)** The given recurrence says that if $n=0$ then $f_0 = 0$ and if $n=1$ then $f_1 = 1$. These two conditions (or values) where recursion does not call itself is called an **initial condition** (or **Base conditions**).
2. **(Recursive step):** This step is used to find new terms $f_2, f_3, \dots$, from the existing (preceding) terms, by using the formula $f_n = f_{n-1} + f_{n-2}$ for $n \geq 2$.

This formula says that "by adding two previous sequence (or term) we can get the next term".

For example $f_2 = f_1 + f_0 = 1 + 0 = 1$;

$f_3 = f_2 + f_1 = 1 + 1 = 2$; $f_4 = f_3 + f_2 = 2 + 1 = 3$ and so on

Example 2 Find out the value of $n! = n(n-1)(n-2)\dots(3)(2)(1)$ for $n \geq 1$

Factorial function is defined as:

$$n! = \begin{cases} 1 & \text{if } n = 1 \\ n \cdot (n-1)! & \text{if } n > 1 \end{cases}$$

Let us write an algorithm for factorial function:

int fact(int n)

```
1: if n == 0 then
2: return 1
3: else
4: return n * (n - 1)
5: endif
```

Let us try to understand the efficiency of the algorithm in terms of the number of multiplications operations required for each value of n

Let (n) denoted the number of multiplication required to execute the $n!$,

that is $T(n)$ denotes the number of times the line 4 is executed in factorial algorithm.

- We have the initial condition $T(0) = 1$; since when $n = 0$, fact simply returns (i.e. Number of multiplication is 0).
- When $n > 1$, the line 4 performs 1 multiplication plus fact is recursively called with input $(n - 1)$. It means, by the definition of (n) , additional $(n - 1)$ number of multiplications are required.

We can write a recurrence relation for the factorial as:

$$T(n) = \begin{cases} 1, & \text{if } n = 0 \\ 1 + (n - 1) \end{cases}$$

Algorithm 3: The following algorithm calculates x^n :

Algorithm 3: *Power* (x, n)

```

1: if( $n == 0$ )
2: return 1
3: if( $n == 1$ )
4: return  $x$ 
5: else
6: return  $x * (x, n - 1)$ ;
7: endif

```

The base case is reached when $n == 0$ or $n == 1$.

The algorithm 3 performs one comparison and one return statement. Therefore,
 (0) and $(1) = 0(1) = a$

When $n > 1$; the **algorithm3** performs one recursive call with input parameter $(n - 1)$ at line 6, and some constant number of basic operations. Thus we obtain the recurrence relation as:

$$(n) = \begin{cases} (1) = a & \text{(base case)} \\ (n - 1) + b & \text{(Recursive step)} \end{cases}$$

Example 4: A customer makes an investment of Rs. 5000 at 15 percent annual compound interest. If T_n denotes the amount earned at the end of n years, define a recurrence relation and initial conditions

Ans- At the end of $n-1$ years, the amount is T_{n-1} . After one more year, the amount will be T_{n-1} + the interest amount. Therefore

$$T_n = T_{n-1} + (15\%)_{-1} = (0.15)T_{n-1} = (1.15)T_{n-1}; n \geq 1$$

To find out the recurrence relation when $n=1$ (base value) we have to find the value of T_0 .

Since T_0 refers to the initial amount, $T_0 = 5000$

With the above definitions we can calculate the value of T_n for any value of n . For example:

$$T_3 = (1.15)_2 = (1.15)(1.15)T_1 = (1.15)(1.15)(1.15)T_0 = (1.15)^3(5000)$$

The above computation can be extended to any arbitrary value of n .

$$T_n = (1.15)_{n-1}$$

.. ..

$$= ((1.15)(5000))$$

4.3 METHODS FOR SOLVING RECURRENCE RELATIONS

Three methods are discussed here to solve recurrence relations: Substitution method, Recursion Tree method and Master method. We start with Substitution method

4.3.1 Substitution Method

Substitution is opposite of induction. We start at n and move backward. A substitution method is one, in which we guess a bound and then use mathematical induction to prove whether our guess is correct or not?. It comprises two steps:

Step1: Guess the asymptotic bound of the Solution.

Step2: Prove the correctness of the guess using Mathematical Induction.

Example 1. Solve the following recurrence by using substitution method.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Solution: step1: The given recurrence is quite similar to that of Merge Sort algorithm, Therefore, our guess to the solution is $(n) = O(n \log n)$

Or $(n) \leq c.n \log n$

Step2: Now we use mathematical Induction.

Here our guess does not hold for $n=1$ because $(1) \leq c.1 \log 1$
i.e. $(n) \leq 0$ which is contradiction with $(1) = 1$

Now for $n=2$

$$(2) \leq c.2 \log 2$$

$$2T\left(\frac{2}{2}\right) + 2 \leq c.2$$

$$2(1) + 2 \leq c.2$$

$$0 + 2 \leq c.2$$

$2 \leq c.2$ which is true. So $(2) \leq c.n \log n$ is True for $n = 2$

So

$(2) \leq c.n \log n$ is True for $n = 2$

(i) **Induction step:** Now assume it is true for $n = n/2$

Now we have to show that it is true for the value of n

$$i.e. (2) \leq c \cdot n \log n$$

We know that $(n) \leq 2T(\lfloor \frac{n}{2} \rfloor) + n$

$$\leq 2(c \lfloor \frac{n}{2} \rfloor \log \lfloor \frac{n}{2} \rfloor) + n$$

$$\leq cn \log \lfloor \frac{n}{2} \rfloor + n \leq cn \log n - cn \log 2 + n$$

$$\leq cn \log n - cn + n$$

$$\leq cn \log n \quad \forall c \geq 1$$

Thus $(n) = (n \log n)$

Remark: Making a good guess, which can be a solution of a given recurrence, requires experiences. So, in general, we are often not using this method to get a solution of the given recurrence.

4.3.2 RECURSION TREEMETHOD

A recursion tree is a convenient way to visualize what happens when a recurrence is iterated. It is a pictorial representation of a given recurrence relation, which shows how Recurrence is divided till Boundary conditions.

Recursion tree method is especially used to solve a recurrence of the form:

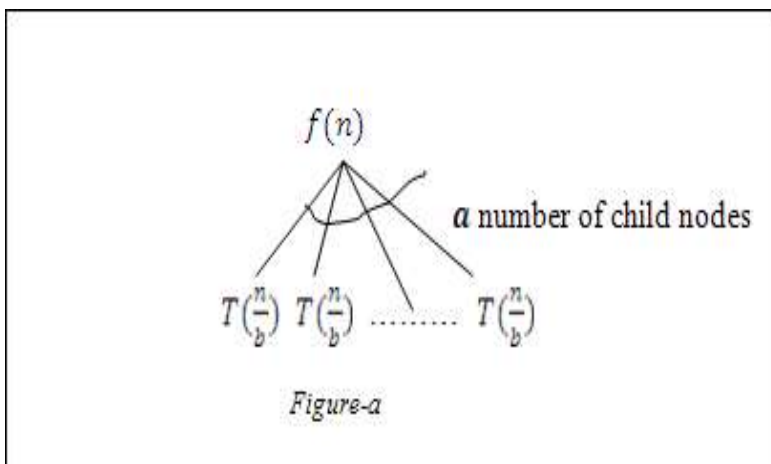
$$T(n) = aT\left(\frac{n}{b}\right) + (n) \dots \dots (1) \quad \text{where } a > 1, b \geq 1$$

This recurrence (1) describe the running time of any divide-and-conquer algorithm.

Method (steps) for solving a recurrence $(n) = aT\left(\frac{n}{b}\right) + (n)$ using recursion tree:

We make a recursion tree for a given recurrence as follows:

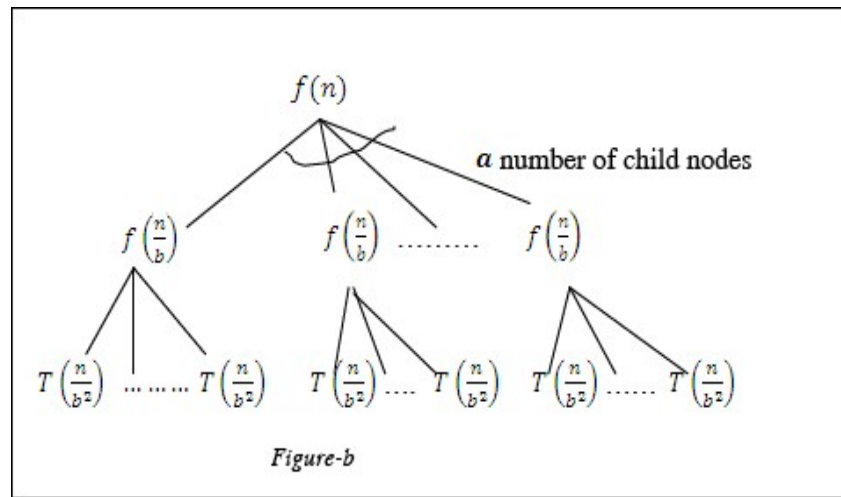
- To make a recursion tree of a given recurrence (1), First put the value of $f(n)$ at root node of a tree and make a ***a number*** of child nodes of this root value $f(n)$
Now tree will be looks like as:



(b) Now we have to find the value of $T\left(\frac{n}{b}\right)$ by putting (n/b) in place of n in equation (1). That is

$$T\left(\frac{n}{b}\right) = aT\left(\frac{n}{b^2}\right) + f(n/b) = aT + f\left(\frac{n}{b^2}\right) + f(n/b) \dots (2)$$

From equation (2), now (n/b) will be the value of node having a branch (child nodes) each of size $T(n/b)$. Now each $T\left(\frac{n}{b}\right)$ in **figure-a** will be replaced as follows:



c) In this way you can expand a tree one more level (i.e. up to (at least) 2 levels).

Step2: (a) Now you have to find per level cost of a tree. Per level cost is the sum of the cost of each node at that level. For example per level cost at level 1 is

$$T\left(\frac{n}{b}\right) + f\left(\frac{n}{b}\right) + \dots + f\left(\frac{n}{b}\right) \text{ (a times). This is also called Row-Sum.}$$

(b) Now the total (final) cost of the tree can be obtained by taking the sum of costs of all these levels.

$$i.e. \text{Total cost} = \text{sum of costs of } (l_0 + l_1 + \dots + l_k).$$

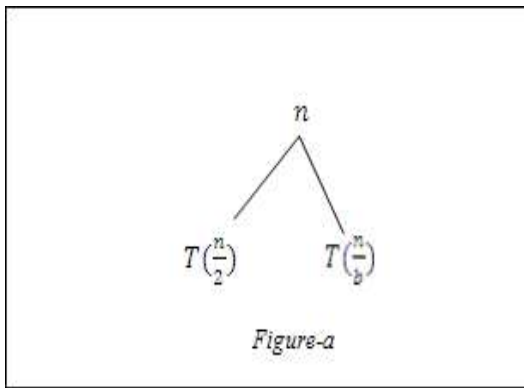
This is also called **Column-Sum**.

Let us take one example to understand the concept to solve a recurrence using recursion tree method:

Example1: Solve the recurrence $T(n) = 2T\left(\frac{n}{2}\right) + n$ using recursion tree method.

Solution: Step1: First you make a recursion tree of a given recurrence.

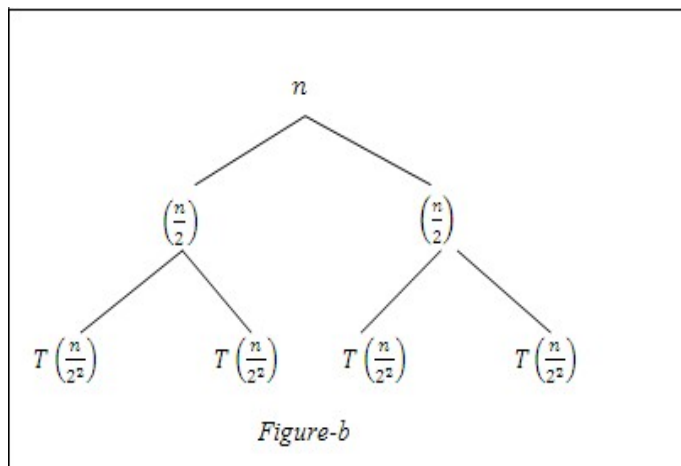
1. To make a recursion tree, you have to write the value of (n) at root node. And
2. The number of child of a Root Node is equal to the value of a . (Here the value of $a = 2$). So recursion tree looks like as:



b) Now we have to find the value of $T\left(\frac{n}{2}\right)$ in figure (a) by putting $(n/2)$ in place of n in equation (1). That is

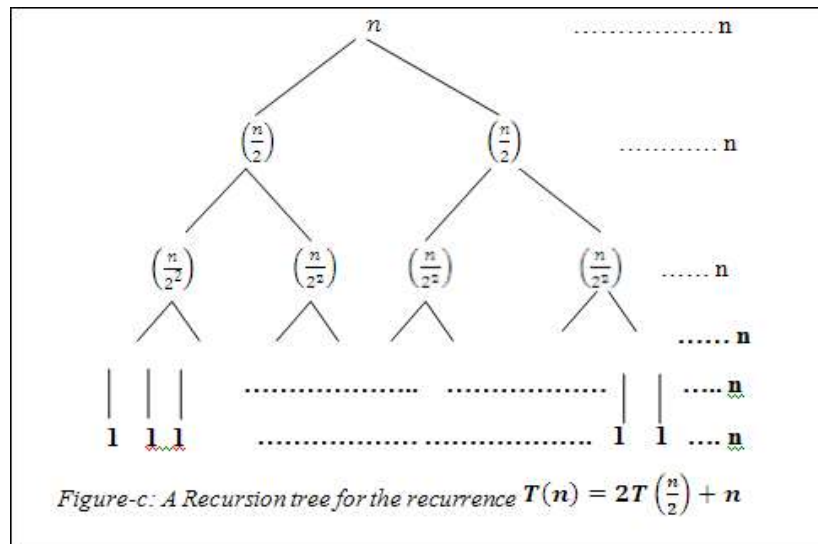
$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \dots (2)$$

From equation (2), now $\left(\frac{n}{2}\right)$ will be the value of node having 2 branch (child nodes) each of size $T(n/2)$. Now each $T\left(\frac{n}{2}\right)$ in **figure-a** will be replaced as follows:



Solving Recurrence

- c) **In this way, you can extend a tree up to Boundary condition (when problem size becomes 1). So the final tree will be looks like:**



Now we find the per level cost of a tree, Per-level cost is the sum of the costs within each level (called row sum). Here per level cost is For example: per level cost at depth 2 in **figure-c** can be obtained as:

$$\left(\frac{n}{2^2}\right) + \left(\frac{n}{2^2}\right) + \left(\frac{n}{2^2}\right) + \left(\frac{n}{2^2}\right) = n.$$

Then total cost is the sum of the costs of all levels (called column sum), which gives the solution of a given Recurrence. The height of the tree is

$$\text{Total cost} = n + n + n + \dots + n \dots \dots (3)$$

To find the sum of this series you have to find the total number of terms in this series. To find a total number of terms, you have to find a height of a tree.

Height of tree can be obtained as follow (see recursion tree of figure c): you start a problem of size n , then problem size reduces to $\left(\frac{n}{2}\right)$, then $\left(\frac{n}{2^2}\right)$, and so on till boundary condition (problem size 1) is not reached. That is

$$n \rightarrow \left(\frac{n}{2}\right) \rightarrow \left(\frac{n}{2^2}\right) \rightarrow \dots \dots \dots \rightarrow \left(\frac{n}{2^k}\right)$$

At last level problem size will be equal to 1 if

$$\left(\frac{n}{2^k}\right) = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n.$$

This k represent the height of the tree, hence height $= k = \log_2 n$.

Hence total cost in equation (3) is

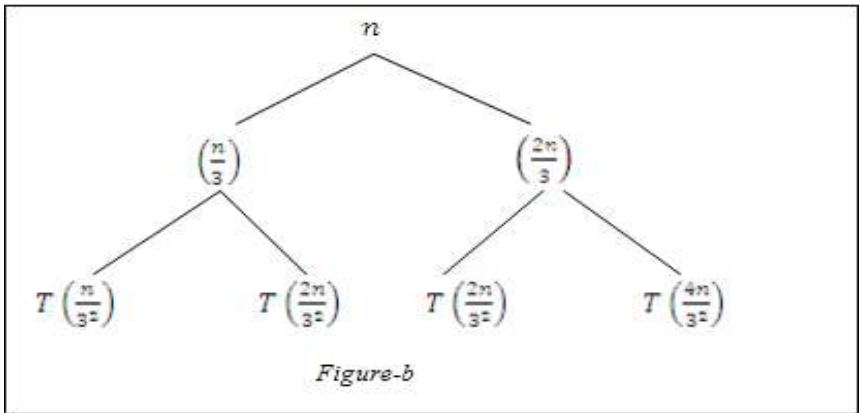
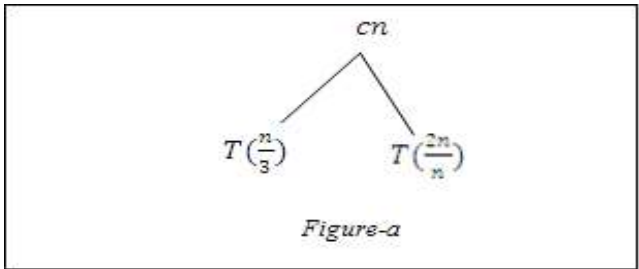
$$n + n + n + \dots + (\log_2 n \text{ terms}) = n \log_2 n \Rightarrow (n \log_2 n).$$

Example2: Solve the recurrence $T(n) = T(\lfloor \frac{n}{3} \rfloor) + T(\lfloor \frac{2n}{3} \rfloor) + n$ using recursion tree method.

Solution: We always omit floor & ceiling function while solving recurrence. Thus given recurrence can be written as:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n \dots \dots (1)$$

Figure-a to figure-c shows a step-by-step derivation of a recursion tree for the given recurrence (1).



c) **In this way, you can extend a tree up to Boundary condition** (when problem size becomes 1). So the final tree will be looks like:

Here the smallest path from root to the leaf is:

$$n \rightarrow \left(\frac{2}{3}\right)n \rightarrow \left(\frac{2}{3}\right)^2 n \rightarrow \dots 1$$

$$\left(\frac{2}{3}\right)^k = 1 \Rightarrow k = \log_{3/2} n \Rightarrow \text{Height of the tree.}$$

$$n + n + \dots \dots \dots + n(\log_{3/2} n \text{ times})$$

$$\Rightarrow n \log_{3/2} n = \frac{n \log_2 n}{\log_2 \frac{3}{2}} = O(n \log_2 n) \dots \dots \dots (*)$$

Here the smallest path from root to the leaf is:

Solving Recurrence

$$n \rightarrow \left(\frac{n}{3}\right) \rightarrow \left(\frac{n}{3^2}\right) \rightarrow \dots \dots \dots \rightarrow \left(\frac{n}{3^k}\right)$$

$$(n/3)^k = 1 \Rightarrow k = \log_3 n \Rightarrow \text{Height of the tree.}$$

$$n + n + \dots \dots \dots + (\log_3 n \text{ times})$$

$$\Rightarrow n \log_3 n = \frac{n \log_2 n}{\log_2 3} = \Omega(n \log_2 n) \dots \dots \dots (**)$$

Form equation (*) and

(**), Since $T(n) = O(n \log_2 n)$ and $T(n) = \Omega(n \log_2 n)$, thus we write:

$$T(n) = \Theta(n \log_2 n)$$

Remark: If

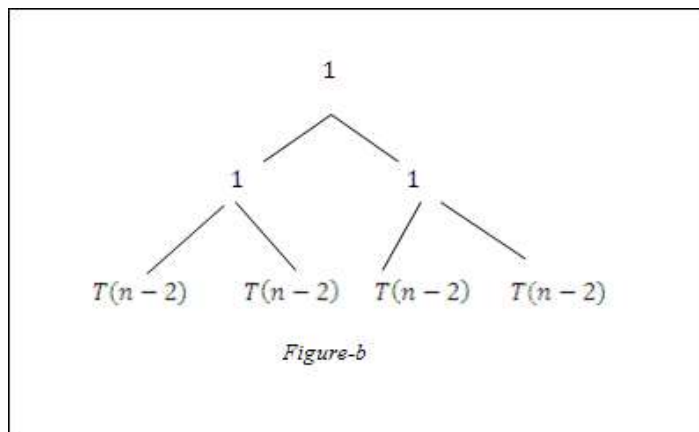
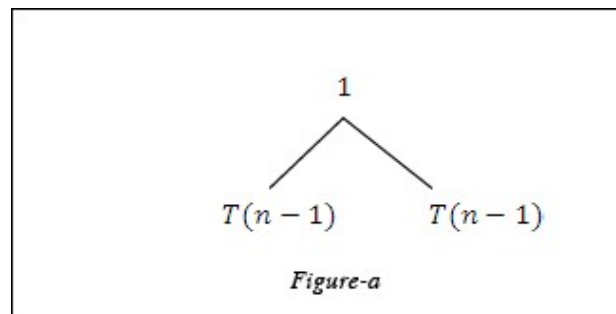
$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)) \text{ then } f(n) = \Theta(g(n))$$

Example3: A recurrence relation for Tower of Hanoi (TOH) problem is

$T(n) = 2T(n-1) + 1$ with $T(1) = 1$ and $T(n) = 3$. Solve this recurrence to find the solution of TOH problem.

Solution:

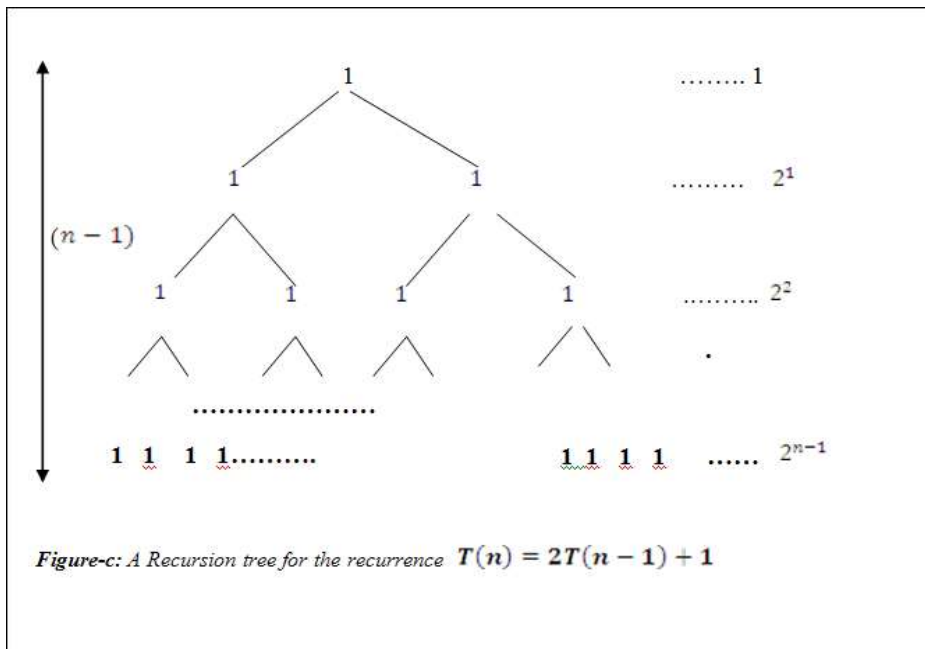
Figure-a to figure-c shows a step-by-step derivation of a recursion tree for the given recurrence $T(n) = 2T(n-1) + 1$



c) In this way, you can extend a tree up to Boundary condition (when problem size becomes 1). That is

$$n \rightarrow (n-1) \rightarrow (n-2) \rightarrow \dots \dots \dots \rightarrow (n-(n-1))$$

$$\Rightarrow n \rightarrow (n-1) \rightarrow (n-2) \rightarrow \dots \dots \dots \rightarrow 2 \rightarrow 1$$



So the final tree will be looks like:

At last level problem size will be equal to 1 if
 $(n - (n-1)) = 1 \Rightarrow \text{Height of the tree} \Rightarrow (n-1)$.

Hence Total Cost of the tree in figure (c) can be obtained by taking column sum upto the height of the tree.

$$T(n) = 1 + 2^1 + 2^2 + \dots \dots + 2^{n-1} = \frac{1(2^n - 1)}{2 - 1} = 2^n - 1.$$

Hence the solution of TOH problem is $T(n) = (2^n - 1)$

☞ Check Your Progress 1

Q1: write a recurrence relation for the following recursive functions:

```
a) Fast_Power(x,n)
    {if (n == 0)
        return 1;
    elseif (n == 1)
        return x;
    elseif ((n%2) == 0) //if n is even
        return Fast_power(x, n/2) * Fast_power(x, n/2);
    else
```

Solving Recurrence

```
return x * Fast_power (x,  $\frac{n}{2}$ ) * Fast_power (x,  $\frac{n}{2}$ )
}
```

.....

.....

.....

.....

b)

Fibonacci (n)

{ if (n == 0)

return 0;

if (n == 1)

return 1;

return fibonacci (n-1) + fibonacci (n-2); }

.....

.....

.....

.....

Q.2: Solve the following recurrence Using Recursion tree method

a. $T(n) = 4T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n$

.....

.....

.....

.....

b. $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n$

.....

.....

.....

.....

Definition 1: A function $f(n)$ is asymptotically positive if and only if there exists a real number n such that $f(x) > 0$ for all $x > n$.

The master method provides us a straight forward method for solving recurrences of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \text{ where } a \geq 1 \text{ and } b > 1 \text{ are constants and } f(n) \text{ is an}$$

asymptotically positive function. This recurrence gives us the running time of an algorithm that divides a problem of size n into a **subproblems** of size $\left(\frac{n}{b}\right)$.

The a **subproblems** are solved recursively, each in time $T\left(\frac{n}{b}\right)$. The cost of dividing the problem and combining the results of the subproblems is described by the function $f(n)$. This recurrence is technically correct only

when $\left(\frac{n}{b}\right)$ is an integer, so the assumption will be made that $\left(\frac{n}{b}\right)$ is either $\left\lfloor \frac{n}{b} \right\rfloor$

or $\left\lceil \frac{n}{b} \right\rceil$ since such a replacement does not affect the asymptotic behavior of the recurrence.

The value of a and b is a positive integer since one can have only a whole number of subproblems.

Theorem1: Master Theorem

The Master Method requires memorization of the following 3 cases; then the solution of many recurrences can be determined quite easily, often without using pencil & paper.

Let $T(n)$ be defined on the non negative integers by:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \text{ where } a \geq 1, b > 1 \text{ and } \frac{n}{b} \text{ is treated as above ----- (1)}$$

Then $T(n)$ can be bounded asymptotically as follows:

Case1: If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \theta(n^{\log_b a})$

Case2: If $f(n) = \theta(n^{\log_b a})$, then $T(n) = \theta(n^{\log_b a} \cdot \log n)$.

Case3: If $f(n) = \theta(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and if $a f\left(\frac{n}{b}\right) \leq c f(n)$ for

some constant $c < 1$ and all sufficiently large n . $T(n) = \theta(f(n))$

Remark: To apply Master method you always compare $n^{\log_b a}$ and $f(n)$. The larger of the two functions determines solution to the recurrence problems. If the growth rate of these two functions then it belongs to **case 2**. In this case we multiply by a logarithmic factor to get the run time solution ($T(n)$) of recurrence relation.

Solving Recurrence

If $f(n)$ is polynomially smaller than $n^{\log_b a}$ (by a factor of n^ϵ) then **case 1** will be applicable to find $T(n)$.

If $f(n)$ is polynomially larger than $n^{\log_b a}$ (by a factor of $1/n^\epsilon$) then $T(n) = \theta(f(n))$ which is in **case 3**.

Examples of Master Theorem

Example1: Consider the recurrence of $T(n) = 9T\left(\frac{n}{3}\right) + n$, in which $a = 9$, $b = 3$, $f(n) = n$, $n^{\log_b a} = n^2$ and $f(n) = O(n^{\log_b a - \epsilon})$, where $\epsilon = 1$. The growth rate of $f(n)$ is slower, we will apply the case 1 of Master Theorem and we get $T(n) = \Theta(n^{\log_b a - \epsilon}) = \Theta(n^2)$

Example2: Consider the recurrence of $T(n) = T\left(\frac{2n}{3}\right) + 1$, in which $a = 1$, $b = \frac{3}{2}$, $f(n) = n^{\log_{3/2} 1} = 1$. Since $f(n) = \Theta(n^{\log_{3/2} 1})$. By Master Theorem (case2), we get $T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(\log n)$.

☞ Check Your Progress 2

Q.1: Write the first two cases (Case 1 and Case 2) of Master method to solve a recurrence relation of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

.....

.....

.....

.....

Q.2: Use Master Theorem to give the tight asymptotic bounds of the following recurrences:

a. $T(n) = 4T\left(\frac{n}{2}\right) + n$

.....

.....

.....

.....

b. $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

4.4 SUMMARY

When an algorithm contains a recursive call to itself, its running time can often be described by a recurrence equation which describes a function in terms of its value on smaller inputs.

There are three basic methods of solving the recurrence relation:

1. The Substitution Method
2. The Recursion-tree Method
3. The Master Theorem

Master method provides a “cookbook” method for solving recurrences of the

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$ where $a \geq 1$ and $b > 1$ are constants from:
 $f(n)$

In master method you have to always compare the value of $f(n)$ with $n^{\log_b a}$ to decide which case is applicable.

4.5 SOLUTIONS/ANSWERS

Check Your Progress 1:

Q 1: a)

At every step the problem size reduces to half the size. When the power is an odd number, the additional multiplication is involved. To find a time complexity of this algorithm, let us consider the **worst case**, that is we assume that at every step additional multiplication is needed. Thus total number of operations $T(n)$ will reduce to number of operations for $n/2$, that is $T(n/2)$ with three additional arithmetic operations (In odd power case: 2 multiplication and one division). Now we can write:

$$T(n) = 1 \text{ if } n = 0 \text{ or } 1$$

$$T(n) = T\left(\frac{n}{2}\right) + 3 \text{ if } n \geq 2$$

Instead of writing exact number of operations needed by the algorithm, we can use some constants. The reason for writing this constant is that we are always interested to find “asymptotic complexity” instead of finding exact number of operations needed by algorithm, and also it would not affect our complexity also.

$$T(n) = \begin{cases} T(1) = a & \text{if } n = 0 \text{ or } n = 1 & \text{(base case)} \\ T\left(\frac{n}{2}\right) + b & \text{if } n \geq 2 & \text{(Recursive step)} \end{cases}$$

Solving Recurrence

$$b) T(n) = \begin{cases} a & \text{if } n = 0 \text{ or } n = 1 \\ T(n-1) + T(n-2) + b & \text{if } n \geq 2 \end{cases}$$

(base case)
(Recursive step)

Q2 (a) The recursion tree for the given recurrence relation is:

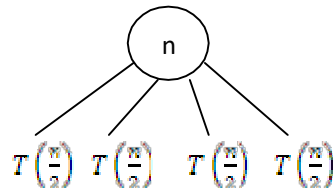


Figure a

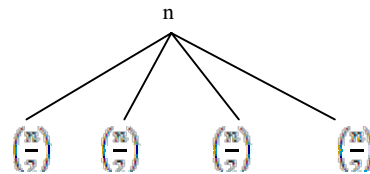


Figure b

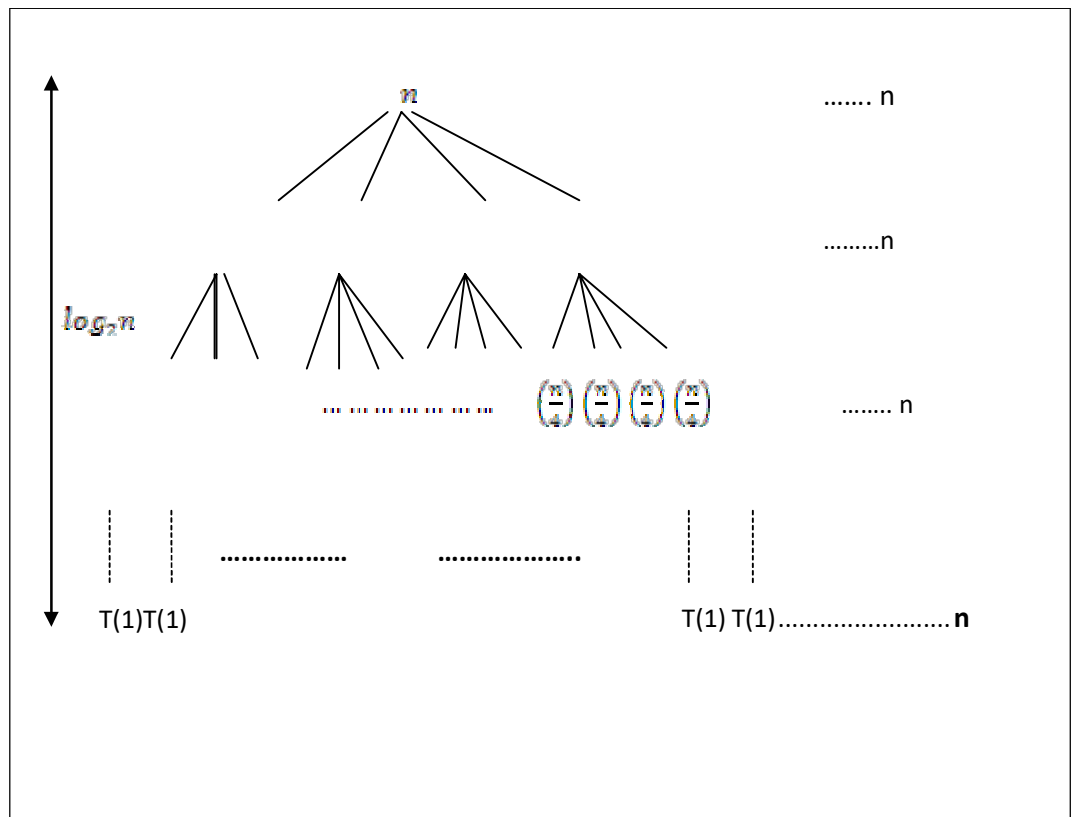


Figure c: A Recurrence Tree for $T(n) = 4T\left(\frac{n}{2}\right) + n$

We have $Total = n + 2n + 4n + \dots \log_2 n \text{ times}$
 $= n(1 + 2 + 4 + \dots \log_2 n \text{ times})$

$$= n \cdot \frac{(2 \log^2 - 1)}{(2)} = n^2 - n = \Theta(n^2)$$

Q2(d)

Check Your Progress 2:

Q1: The following 3 cases are used to solve a recurrence

Case1: If for some $(n) = \Theta(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$

Case2: If $(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.

Solution2:

a) In a recurrence $(n) = 4T\left(\frac{n}{2}\right) + n$, $a = 4, b = 2$,
 $(n) = n^{\log_b a} = n^2$. Now compare $f(n)$ with $n^{\log_b a}$.
 Since $f(n) = O(n^{\log_b a - \epsilon})$.
 where $\epsilon = 1$. By Master Theorem case 1 we get $(n) = \Theta(n^2)$.

b) $(n) = 4T\left(\frac{n}{2}\right) + n^2$; in which $a = 4, b = 2$, $f(n) = n^2$.

Now compare $f(n)$ with $n^{\log_b a}$;
 since $(n) = n^2 = \Theta(n^{\log_b a})$.
 Thus By Master Theorem (case2),
 we get $(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n^2 \log n)$.